KubeVirt Deployment Guide

Deploy Migrated VMs to Any Kubernetes Cluster

K3s, vanilla Kubernetes, EKS, GKE, AKS, OpenShift — hyper2kvm deploys to all of them. This guide covers KubeVirt installation, CDI setup, PVC upload, VirtualMachine manifests, GitOps fleet management, multi-cluster deployment, and storage options. Run VMs and containers side by side on one platform.

Any K8s | K3s Single-Node | GitOps Fleet | CDI Upload | Dual Target | Proprietary (HyperSDK)

What Is KubeVirt?

Run virtual machines as Kubernetes resources — same APIs, same tooling, same cluster as your containers.

VMs as K8s Resources

- VirtualMachine CRD = VM definition
- VirtualMachineInstance = running VM
- Managed by kubectl, Helm, ArgoCD
- Scheduled like pods — same node selector, affinity, tolerations
- Lifecycle: start, stop, restart, migrate, snapshot

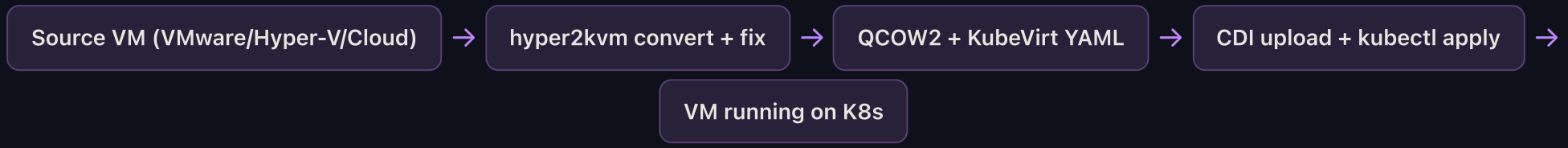
Why KubeVirt?

- Converge VMs + containers on one platform
- GitOps: VM definitions in Git, ArgoCD deploys
- Same monitoring (Prometheus) for VMs and pods
- Same networking (Calico, Cilium, Multus)
- Path to containerization — run VM while rebuilding as container

Where It Runs

- **K3s** — single node, 512MB RAM, edge
- **Vanilla K8s** — kubeadm, kubespray
- **EKS/GKE/AKS** — managed K8s (bare-metal nodes)
- **OpenShift** — as OpenShift Virtualization
- **Rancher/Harvester** — HCI platforms

hyper2kvm KubeVirt Pipeline



Supported Kubernetes Distributions

Distribution	KubeVirt	CDI	Min Nodes	Best For	hyper2kvm Tested
K3s	Yes	Yes	1	Edge, dev, small sites	**Verified (Photon OS)**
Vanilla K8s	Yes	Yes	1	General purpose	**Verified**
kubeadm	Yes	Yes	1	Self-managed clusters	**Verified**
OpenShift	Yes (OCP Virt)	Yes	3+3	Enterprise Red Hat shops	**Verified**
EKS (bare-metal)	Yes	Yes	1+	AWS with bare-metal nodes	**Compatible**
Rancher/RKE2	Yes	Yes	1	Multi-cluster management	**Compatible**

Harvester	**Built-in**	**Built-in**	1	HCI (hyperconverged)	**Compatible**

K3s + KubeVirt Quick Setup

From bare Linux to running KubeVirt VMs in 10 minutes.

Step 1: Install K3s (2 minutes)

```
# Single-node K3s — one command
curl -sfL https://get.k3s.io | sh -

# Verify
sudo kubectl get nodes    # Node should be Ready
sudo kubectl get pods -A  # All system pods running
```

Step 2: Install KubeVirt + CDI (3 minutes)

```
# Install KubeVirt operator
export KUBEVIRT_VERSION=$(curl -s https://api.github.com/repos/kubevirt/kubevirt/releases/latest | grep tag_name | cut -d'"' -f4)
sudo kubectl create -f https://github.com/kubevirt/kubevirt/releases/download/$KUBEVIRT_VERSION/kubevirt-operator.yaml
sudo kubectl create -f https://github.com/kubevirt/kubevirt/releases/download/$KUBEVIRT_VERSION/kubevirt-cr.yaml

# Install CDI (Containerized Data Importer) for disk upload
export CDI_VERSION=$(curl -s https://api.github.com/repos/kubevirt/containerized-data-importer/releases/latest | grep tag_name | cut -d'"' -f4)
sudo kubectl create -f https://github.com/kubevirt/containerized-data-importer/releases/download/$CDI_VERSION/cdi-operator.yaml
sudo kubectl create -f https://github.com/kubevirt/containerized-data-importer/releases/download/$CDI_VERSION/cdi-cr.yaml

# Install virtctl CLI
sudo curl -L -o /usr/local/bin/virtctl https://github.com/kubevirt/kubevirt/releases/download/$KUBEVIRT_VERSION/virtctl-$KUBEVIRT_VERSION-linux-amd64
sudo chmod +x /usr/local/bin/virtctl

# Wait for ready
sudo kubectl wait -n kubevirt kv/kubevirt --for=condition=Available --timeout=300s
```

Step 3: Deploy VM with hyper2kvm (5 minutes)

```
# Convert VMDK and deploy to KubeVirt in one command
cat > kubevirt-deploy.yaml <<'EOF'
cmd: local
vmdk: ./photon-disk1.vmdk
output_dir: ./converted
out_format: qcow2
regen_initramfs: true
deploy_k8s: true              # Generate KubeVirt manifest + upload
k8s_namespace: default
```

```
vm_name: photon-kv
memory: 1024
vcpus: 2
EOF

sudo h2kvmctl --config kubevirt-deploy.yaml

# Verify
sudo kubectl get vm                  # VirtualMachine defined
sudo kubectl get vmi                 # VirtualMachineInstance running
sudo virtctl console photon-kv       # Connect to console
```

Generated KubeVirt Manifest (auto-created by hyper2kvm)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: photon-kv
  namespace: default
spec:
  running: true
  template:
    spec:
      domain:
        resources:
          requests:
            memory: 1Gi
            cpu: "2"
        devices:
          disks:
          - name: rootdisk
            disk:
              bus: virtio
          interfaces:
          - name: default
            masquerade: {}
      networks:
      - name: default
        pod: {}
      volumes:
      - name: rootdisk
        persistentVolumeClaim:
          claimName: photon-kv-disk
```

GitOps Fleet Management

Manage 100+ edge sites from one Git repo. Push a commit, deploy to every cluster.

ArgoCD / Flux Workflow

- Convert VMs at HQ with hyper2kvm
- Upload disk images to container registry
- Store KubeVirt manifests in Git repo
- ArgoCD/Flux watches repo, syncs to all clusters
- One git push = deploy to 100 edge K3s clusters
- Rollback by reverting a commit
- Full audit trail in Git history

Multi-Cluster Architecture

- **Hub cluster:** ArgoCD + registry at HQ
- **Spoke clusters:** K3s at each edge site
- WireGuard/Tailscale VPN between sites
- Each site auto-joins fleet on boot
- Namespace per site or per workload type
- Prometheus federation for central monitoring
- Zero SSH — all management via K8s API

Git Repository Structure

```
fleet-vms/
  base/                          # Shared VM templates
    photon-vm.yaml               # Base VirtualMachine manifest
    ubuntu-vm.yaml
    windows-vm.yaml
  overlays/
    site-nyc/                    # Per-site customizations
      kustomization.yaml
      patch-resources.yaml       # Memory/CPU overrides for this site
    site-london/
      kustomization.yaml
    site-tokyo/
      kustomization.yaml
  argocd/
    applicationset.yaml          # Deploy to all clusters matching label
```

Storage Options for KubeVirt Disks

Storage	Type	Best For	Performance	Cost
local-path	Local NVMe/SSD	Single-node K3s, edge	**Best (direct I/O)**	**$0 (included)**
Longhorn	Distributed block	Multi-node, HA, replication	Good	**$0 (CNCF)**

Ceph/Rook	Distributed block/fs	Large clusters, enterprise HA	Good	**$0 (open source)**
OpenEBS	Container-attached	Cloud-native storage	Good	**$0 (CNCF)**
NFS	Network filesystem	Shared storage, simple	Moderate	**$0**
AWS EBS CSI	Cloud block	EKS bare-metal nodes	Good	$$

Dual-Target: KVM + KubeVirt from Same Conversion

```
# One conversion, two deployment targets
cmd: vsphere
vs_action: export_vm
vs_vm: production-app-01
output_dir: ./converted
out_format: qcow2
regen_initramfs: true
emit_domain_xml: true     # Target 1: libvirt XML for standalone KVM
deploy_k8s: true           # Target 2: KubeVirt manifest for K8s
k8s_namespace: production
vm_name: app-01
memory: 4096
vcpus: 4
```

Dual target: hyper2kvm generates both libvirt domain XML and KubeVirt VirtualMachine YAML from a single conversion. Deploy the right format per site — KVM for traditional hosts, KubeVirt for K8s clusters. No reconversion needed.

Advanced KubeVirt Features

Enterprise capabilities for production KubeVirt deployments.

Live Migration

- Move running VMs between nodes — zero downtime
- Automatic during node drain (kubectl drain)
- Requires shared storage (Ceph, Longhorn, NFS)
- Configurable bandwidth limits
- `virtctl migrate vm-name`

Snapshots & Backup

- VolumeSnapshot for point-in-time disk copies
- Velero integration for cluster-level backup
- Snapshot before migration for rollback safety
- Clone VMs from snapshots for testing
- CSI driver required (Ceph, Longhorn support it)

Networking

- **Masquerade:** Default, NAT through pod network
- **Bridge:** L2 connectivity, VLAN support
- **SR-IOV:** Near-native NIC performance
- **Multus:** Multiple NICs per VM
- Same CNI as containers (Calico, Cilium)

GPU & Special Devices

- GPU passthrough (NVIDIA, AMD) for AI/VDI
- vGPU (NVIDIA GRID) for shared GPU
- USB passthrough for hardware dongles
- TPM emulation (swtpm) for Windows 11
- Host device passthrough via VFIO

Windows on KubeVirt

- Windows Server 2016-2025 fully supported
- Windows 11 with TPM emulation
- VirtIO drivers auto-injected by hyper2kvm
- UEFI + Secure Boot via OVMF
- RDP access via NodePort or LoadBalancer service

Monitoring & Operations

- Prometheus metrics for VM CPU/RAM/disk/network
- Grafana dashboards (community templates)
- kubectl top vm for resource usage
- Alerting on VM health via Alertmanager
- zkvm TUI tab 5: KubeVirt VM management

KubeVirt: The VM-Container Convergence Platform

Run VMs and containers on the same cluster. Manage both with kubectl.
GitOps deploys VMs like it deploys containers. Same monitoring, same networking.

hyper2kvm converts any VM and deploys it to any Kubernetes — automatically.